



Medical Clinic Digital Twin

BLE-Based Real-Time Patient Tracking & Workflow Analytics

Team: Central Link

Supervisor: Prof. Tallal El-Shabrawy

Members: Khalid Ashmawy · Ahmed Khedr · Abdalla Mohamed · Ahmed Farag · Abdullah Sherif

Institution: IoT Elective (2026)

Table of Contents

-  Medical Clinic Digital Twin
 - BLE-Based Real-Time Patient Tracking & Workflow Analytics
 -  Table of Contents
 - Project Overview
 - Problem Statement
 - Proposed Solution
 - System Architecture
 - A. Perception Layer — End Devices
 - B. Edge Layer — Gateways (ESP32 Routers)
 - C. Network Layer — Secure Communication
 - D. Processing Layer — Cloud Backend
 - E. Application Layer — Dashboard
 - Hardware Components
 - Communication Protocols
 - System Workflow
 - Cloud Infrastructure
 - Backend API
 - Authentication
 - Hospital Settings
 - Router (Gateway) Management
 - Device (BLE Tag) Management
 - Patient Tracking
 - Analytics & Records
 - Dashboard & Frontend
 - Dashboard (Home)
 - Router Management
 - Device Management
 - Patient Tracking
 - Analytics
 - Settings
 - Database Design
 - Security
 - Localization Engine

- [Analytics](#)
 - [Workflow Analytics](#)
 - [Related Repositories](#)
 - [Academic References](#)
-

Project Overview

The **Medical Clinic Digital Twin** is a fully integrated IoT-based smart healthcare monitoring system that creates a real-time virtual mirror of a clinic's physical environment. Using Bluetooth Low Energy (BLE) wearable patient tags and ESP32 gateway nodes, the system continuously tracks patient movement across clinic zones, transmits data securely to the cloud, and visualizes live occupancy, wait times, and bottlenecks through a web dashboard.

The system is named **Central Link**, reflecting its role as a unified hub connecting edge hardware, a cloud backend, and a real-time dashboard into a single cohesive digital twin platform.

Problem Statement

In busy medical environments, patient flow management is a persistent operational challenge. The core issue is **operational blindness** — administrators have no real-time visibility into where patients are, how long they've been waiting, or where bottlenecks are forming.

Consequences of the current state:

- Waiting rooms overflow while clinical staff remain underutilized in other areas
- Long Patient Turnaround Time (PTAT) negatively affects health outcomes
- Manual check-in systems introduce significant human error (estimated at ~22.5% inaccuracy per related literature) and cannot capture real-time movement patterns
- Delayed identification of congested zones prevents timely staff reallocation

Existing technology limitations:

- **RFID (Passive):** Prohibitively high deployment cost and very short detection range
 - **GPS / UWB:** Ineffective indoors due to signal attenuation through walls and ceilings
 - **Wi-Fi fingerprinting:** Computationally expensive and energy-intensive for battery-constrained wearables
 - **Manual tracking:** Fundamentally incapable of real-time granularity
-

Proposed Solution

Central Link addresses operational blindness by deploying a tiered IoT architecture that automates data collection from the moment a patient enters the facility. Each patient is issued a lightweight BLE wearable tag upon arrival. Stationary ESP32 gateways installed at key clinic zones scan for these tags, measure signal strength (RSSI), and forward structured telemetry to a cloud server over a secure MQTT connection.

Key differentiators of the Central Link approach:

- **Live Digital Twin:** A 2D blueprint map of the clinic is rendered on the dashboard, showing real-time patient positions resolved to room-level accuracy
 - **Secure MQTTS:** All telemetry is encrypted using TLS 1.2/1.3 certificates, meeting HIPAA-grade privacy requirements
 - **Scalability:** The BLE advertising-only mode of patient tags ensures extremely low power consumption, making the wearables practical for continuous clinical use
-

System Architecture

The system follows a five-layer IoT architecture:

A. Perception Layer — End Devices

BLE wearable tags are assigned to each patient upon arrival. Operating in advertising-only mode, each tag periodically broadcasts a unique patient identifier. This passive broadcast mode maximizes battery life, as the tags never establish a connection — they simply emit their presence.

B. Edge Layer — Gateways (ESP32 Routers)

Stationary ESP32 microcontrollers are deployed at doorways and zone boundaries across the clinic (reception, waiting area, examination rooms, nurse station, pharmacy). Each gateway continuously scans for BLE advertising packets, captures the tag's unique ID, measures the Received Signal Strength Indicator (RSSI), and appends a gateway ID and local timestamp before forwarding the data upstream.

C. Network Layer — Secure Communication

Data travels from the ESP32 gateways to the cloud over Wi-Fi using MQTT over TLS (MQTTS) on port 8883. MQTT's lightweight publish-subscribe model is ideal for real-time IoT telemetry, while TLS ensures the data stream is encrypted end-to-end. Payloads are structured in JSON format for easy parsing by both edge devices and the cloud backend.

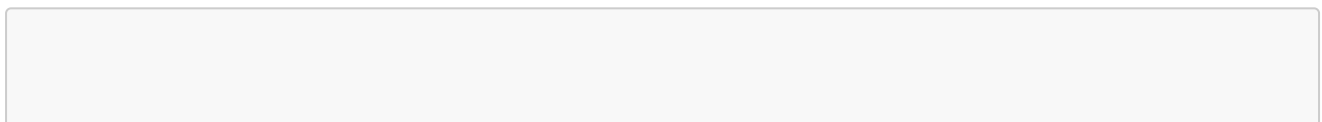
D. Processing Layer — Cloud Backend

An Ubuntu VM on Oracle Cloud hosts the Node.js backend server, the Eclipse Mosquitto MQTT broker, and the Oracle SQL database. This layer is responsible for receiving telemetry from all gateways, resolving patient zone conflicts (when multiple gateways detect the same tag simultaneously), maintaining session records, running analytics queries, and serving processed data to the dashboard via a REST API.

E. Application Layer — Dashboard

A Python Dash / Flask web application serves as the user-facing interface. It fetches processed data from the backend API and renders a live clinic floor plan with patient locations, occupancy statistics, wait-time charts, and anomaly alerts.

Full data flow:



BLE Tag → ESP32 Gateway → MQTT Broker (port 8883, TLS) → Node.js Backend → Oracle DB → Dashboard

Hardware Components

Component	Role	Interface
BLE Wearable Tags	Worn by patients; broadcast a unique UUID as the primary data source	BLE Advertising (2.4 GHz)
ESP32 Microcontrollers	Installed at doorways; scan BLE packets, measure RSSI, and act as MQTTS gateways	BLE Scanner + Wi-Fi
Ubuntu Cloud VM (Oracle)	Hosts the MQTT broker, Node.js backend, and database	MQTTS / HTTPS / REST
Web Dashboard	Accessible via PC or tablet; visualizes clinic layout and analytics	HTTP / WebSockets

Communication Protocols

Link	Protocol	Key Properties
Tag → Gateway	BLE Advertising	Low power, connectionless, passive broadcast
Gateway → Cloud	MQTT over TLS (port 8883)	Lightweight, real-time, encrypted
Data Encryption	TLS 1.2 / 1.3	End-to-end encryption, certificate-based authentication
Cloud → Dashboard	WebSockets / HTTPS	Live updates, secure transport
Payload Format	JSON	Structured, human-readable, easily parsed

System Workflow

The end-to-end operation of the Medical Clinic Digital Twin follows a structured sequence:

- Tag Broadcast** — Upon arrival, a patient is issued a BLE tag. The tag continuously broadcasts advertising packets containing its unique UUID.
- Edge Detection** — As the patient moves through the facility, gateway ESP32 nodes at zone boundaries capture the advertising packets.
- Local Processing** — Each gateway measures the RSSI of the detected tag and attaches its own gateway ID and a local timestamp.

4. **Cloud Transmission** — The gateway publishes the structured JSON payload to the Mosquitto MQTT broker over MQTTS (port 8883).
5. **Data Reconciliation** — The Node.js backend subscribes to all MQTT topics. When the same tag is detected by multiple gateways simultaneously (overlap zones), the system selects the gateway reporting the **strongest RSSI and earliest timestamp** to determine the patient's definitive zone.
6. **Analysis & Visualization** — The database is updated with the resolved patient location, and session durations are calculated.
7. **User Output** — The dashboard displays real-time locations on the clinic floor plan, along with wait-time analytics.

Cloud Infrastructure

The cloud backbone runs on an **Oracle Cloud Ubuntu VM**, configured end-to-end as follows:

- **MQTT Broker:** Eclipse Mosquitto, configured for both plaintext (port 1883, internal testing) and TLS-secured (port 8883, production) operation. Authentication is enforced via a password file; anonymous connections are rejected.
- **TLS Certificates:** Initially self-signed certificates were generated for development. For production, Let's Encrypt certificates (via Certbot) were provisioned under the domain central-link-iot.duckdns.org, with an auto-renewal hook to restart Mosquitto upon certificate renewal.
- **Reverse Proxy:** Nginx is configured to proxy HTTP/HTTPS traffic — API requests (</api/>) are forwarded to the Node.js backend (port 3000), and dashboard requests (</>) are forwarded to the Gunicorn/Dash frontend (port 8050).
- **Process Management:** PM2 is used to keep both the Node.js backend and the Python frontend running as persistent background services, with automatic restart on failure and system startup.
- **Live Domain:** <https://central-link-iot.duckdns.org>

Backend API

The Node.js backend exposes a structured REST API that serves as the data layer between the database and the dashboard. All endpoints are prefixed under the live domain.

Authentication

Method	Endpoint	Description
POST	/auth/signup	Register a new hospital with admin credentials
POST	/auth/login	Authenticate and receive a session token

Hospital Settings

Method	Endpoint	Description
GET	/settings	Retrieve hospital profile (name, address, admin info)
PUT	/settings	Update hospital profile fields

Method	Endpoint	Description
DELETE	/settings/records	Delete all patient records and session history
DELETE	/settings/account	Permanently delete the hospital account
PUT	/settings/blueprint	Upload a clinic floor plan image
GET	/settings/blueprint	Retrieve the current floor plan image URL

Router (Gateway) Management

Method	Endpoint	Description
POST	/routers	Register a new ESP32 gateway with its floor plan coordinates
GET	/routers	List all registered gateways
GET	/routers/map	Get gateway positions and connected device counts for the floor plan overlay
GET	/routers/active	List gateways that have transmitted data within the active interval
GET	/routers/{id}	Get details of a specific gateway
GET	/routers/{id}/devices	List all devices currently connected to a gateway
GET	/routers/{id}/hourly-sessions-duration	Get hourly session counts and average durations for a gateway

Device (BLE Tag) Management

Method	Endpoint	Description
POST	/devices	Register a new BLE device
GET	/devices	List all registered devices and their assignment status
GET	/devices/with-routers-info	List devices enriched with their current gateway location
PUT	/devices/{id}/release	Unassign a device from its current patient (discharge)

Patient Tracking

Method	Endpoint	Description
POST	/patients	Admit a new patient and assign them a BLE device
GET	/patients	List all active patients
GET	/patients/{id}/sessions	Retrieve the full movement session history for a patient

Analytics & Records

Method	Endpoint	Description
GET	/records/hourly-records	Number of detection records created per hour
GET	/records/hourly-patients	Number of unique active patients per hour
GET	/records/hourly-sessions-duration	Average total session duration per hour across the clinic

Dashboard & Frontend

The frontend is a **Python Dash** application served via **Gunicorn** and **Flask**, accessible through the Nginx reverse proxy. It is organized into the following pages:

Dashboard (Home)

The main overview page shows three live KPI cards — active routers, active devices, and network load — above a **Clinic Floor Plan** canvas. The floor plan renders the uploaded blueprint as a background, with colored dot markers representing the last-known position of each active patient tag, updated at a configurable polling interval.

Router Management

Administrators can add new ESP32 gateways by clicking a location on the miniature floor plan to set coordinates, then supplying a router ID and name. Each registered router is displayed as a card showing its online/offline status, connected device count, signal quality, and a bar chart of hourly session activity. Clicking a router card opens a detailed modal with its connected device list and session history.

Device Management

Shows all registered BLE devices with their assignment status (Free / Assigned), the patient currently holding the device, and the last detection timestamp. Devices can be released from patients upon discharge. New devices can be registered via a modal form.

Patient Tracking

Displays a live map of device clusters overlaid on the clinic blueprint, along with an Active Sessions table showing each device, its current router location, the assigned patient name, and time since last update. A side panel allows staff to admit new patients and assign available devices. Clicking a device in the table opens a session detail view including a bar chart of time spent at each router zone.

Analytics

A data-rich page with three time-series charts (hourly records, active devices per hour, average session duration), four headline metric tiles (total active nodes, data throughput, system latency, network load), and a full Tracked Active Users table exportable to CSV. All charts are sourced from live API endpoints.

Settings

Allows administrators to update the hospital profile, upload or replace the clinic blueprint image, reset the admin password, wipe patient records (while preserving router/device configuration), and permanently delete the account.

Database Design

The database is hosted on **Oracle Autonomous Database** and accessed from the cloud VM via Oracle Wallet files and OCI credentials. The schema is structured around five core entities:

- **hospitals** — The top-level tenant entity. Each hospital has a unique `hospital_id`, profile fields, hashed password, and an optional blueprint image path.
- **patients** — Records of admitted patients, each linked to a hospital via foreign key.
- **routers** — Registered ESP32 gateways with their `(location_x, location_y)` coordinates on the floor plan. Unique per hospital by name.
- **devices** — BLE tags registered to the hospital. Each device may be linked to a patient (assigned) or left unlinked (free).
- **records** — The core time-series table. Each row represents a single detection event: a device ID, the router that detected it, an RSSI value, and a timestamp. This table is the foundation for all session tracking, wait-time calculations, and analytics queries.

Session boundaries and wait durations are derived from the records table by grouping consecutive detections by device and router, using the earliest timestamp per detection cluster to define session start times and the gap between clusters to calculate session durations.

Security

Security was treated as a first-class concern throughout the architecture:

- **Transport Encryption:** All gateway-to-cloud communication uses MQTT over TLS 1.2/1.3 on port 8883. Plain MQTT (port 1883) is restricted to internal use only.
 - **Certificate Management:** Production TLS certificates are issued by Let's Encrypt via Certbot for the domain `central-link-iot.duckdns.org`. Certificates auto-renew and trigger a Mosquitto restart via a renewal hook.
 - **Broker Authentication:** Anonymous MQTT connections are disabled. Gateways authenticate with a username/password pair stored in a Mosquitto password file with restricted file permissions.
 - **HTTPS:** The Nginx reverse proxy enforces HTTPS for all dashboard and API traffic. HTTP requests are redirected to HTTPS.
 - **Database Credentials:** Oracle Wallet files (mTLS) are used for database connectivity. OCI configuration and PEM keys are stored outside the web root with restricted permissions.
 - **API Authentication:** The backend implements session-based authentication. All protected endpoints verify the caller's session token before serving data.
-

Localization Engine

The zone resolution logic is the core algorithmic component of the system. Because BLE signals propagate through walls, a single patient tag may be simultaneously detected by two or more adjacent gateways.

Resolution algorithm:

1. When the backend receives records for the same device from multiple routers within a short time window, it enters the reconciliation phase.
2. The router reporting the **highest RSSI** (strongest signal, indicating physical proximity) is selected as the patient's current zone.
3. In a tie-breaking scenario, the router with the **earliest timestamp** is preferred, reflecting the direction of movement.
4. The resolved zone is stored as the canonical location for that device at that time.

This approach provides room-level accuracy without requiring complex trilateration or signal fingerprinting databases, keeping the system computationally lightweight and easy to calibrate.

Analytics

Workflow Analytics

The analytics engine computes the following metrics from the records table, exposed via dedicated API endpoints and visualized on the dashboard:

- **Hourly record counts** — Volume of detection events per hour, used to identify peak activity periods
- **Active patients per hour** — Unique patient count per hour, a proxy for clinic load
- **Average session duration** — Mean time a patient spends at any single zone per hour, used to detect slow zones
- **Router-level hourly sessions** — Per-gateway breakdown of session counts and durations, enabling identification of specific bottleneck locations

Related Repositories

Repository	Description	Link
Frontend	Python Dash / Flask dashboard application with real-time floor plan, analytics, and patient tracking	(placeholder)
Routers	Python firmware for ESP32 gateways — BLE scanning, RSSI capture, and MQTTS publishing	(placeholder)
Devices	Arduino/C++ firmware for ESP32-based BLE wearable tag — advertising UUID broadcast	(placeholder)
Backend	Node.js REST API server — MQTT subscriber, Oracle DB integration, session engine, and analytics queries	(placeholder)

Local Setup

Device Setup

- Open clinic-iot-devices/ble_sender/ble_sender.ino in Arduino IDE
- Install BLE libraries as in tutorial slides
- Compile and upload to BLE device

Routers Setup

- Open clinic-iot-routers in VScode
- Create virtual environment
- Install requirements.txt, using the command `pip install -r requirements.txt`
- Rename .env.example to .env
- Run the `python3 main.py` file

OracleDB Setup

- Create new Oracle database
- Setup database using `clinic-iot-deployment/database-setup.sql`

MQTT Setup

- Install Mosquitto MQTT Broker
- Configure MQTT Basic Port 1883
- Create MQTT username and password
- Start Mosquitto

The steps are in details in the file `clinic-iot-deployment/VM Config.md`

Backend Setup

- Open clinic-iot-backend in VScode
- Install dependencies using the command `npm install`
- Rename .env.example to .env
- Fill MQTT broker and Oracle DB .env variables
- Run `node server.js`

Frontend Setup

- Open clinic-iot-frontend in VScode
- Install requirements.txt, using the command `pip install -r requirements.txt`
- Rename .env.example to .env
- Run `python3 app.py`

Academic References

Authors	Title	Source
---------	-------	--------

Authors	Title	Source
Arumugam, G. R., & Muthaiyah, S. (2024)	A Proof of Concept Using BLE to Optimize Patients' Turnaround Time	IJISRT, 9(5)
Abdulmalek, S., et al. (2022)	IoT-Based Healthcare-Monitoring System towards Improving Quality of Life: A Review	Healthcare (Basel)
Ullah, S., et al. (2025)	Advancing Indoor Positioning Systems: Innovations, Challenges	Robotica
Hailu, T. G., Guo, X., & Si, H. (2025)	Indoor Positioning Systems as Critical Infrastructure	Sensors
Yao, W., Chu, C. H., & Li, Z. (2010)	The Use of RFID in Healthcare: Benefits and Barriers	Research Database
Grieves, M., & Vickers, J. (2017)	Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems	Springer, Cham

Central Link · Medical Clinic Digital Twin · 2026

Supervised by Prof. Tallal El-Shabrawy